

PYTHON AVANZADO

MCP · Agentes IA · Orquestación

Todo lo que necesitas saber **más allá de la sintaxis básica**: decoradores, Pydantic, asyncio, ThreadPoolExecutor, ProcessPoolExecutor, frameworks de orquestación y stacks de producción para construir sistemas de agentes IA y servidores MCP en Python.

#	Sección	Temas clave
1	Decoradores	wraps, parámetros, stacking, clases como decoradores
2	Pydantic v2	BaseModel, validators, discriminated unions, Settings
3	asyncio	event loop, coroutines, Task, gather, Queue, Semaphore
4	ThreadPoolExecutor	I/O-bound, as_completed, contextvars, patrones
5	ProcessPoolExecutor	CPU-bound, shared memory, serialización
6	Patrones avanzados	Context Managers, Protocols, TypeVar, Generics
7	MCP en Python	FastMCP, tools, resources, prompts, transport
8	Frameworks agentes	LangChain, LlamaIndex, Pydantic AI, CrewAI, AutoGen
9	LLM Gateway	LiteLLM, Helicone, Portkey — proxy y cost tracking
10	Stack de producción	structlog, tenacity, APM, Docker, env management

01

Decoradores El meta-lenguaje de Python

¿Qué es un decorador?

Un decorador es una función (o clase) que recibe otra función como argumento y devuelve un callable modificado. Se aplica con la sintaxis **@nombre** justo antes de la definición. En el ecosistema MCP y agentes IA son fundamentales porque frameworks como FastMCP, LangChain o Pydantic AI los usan para registrar herramientas, rutas y validaciones.

1.1 Decorador básico con functools.wraps

```
import functools
import time

def timer(func):
    """Mide tiempo de ejecución preservando metadata."""
    @functools.wraps(func) # preserva __name__, __doc__, __annotations__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} tardó {elapsed:.4f}s")
        return result
    return wrapper

@timer
def fetch_data(url: str) -> dict:
    return {"url": url}
```

■ Siempre usa `@functools.wraps(func)` en el wrapper interno para que inspect, Pydantic y los LLM frameworks lean correctamente la firma y docstring.

1.2 Decorador con parámetros (factory)

```
def retry(max_attempts: int = 3, delay: float = 1.0):
    def decorator(func):
        @functools.wraps(func)
        async def async_wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return await func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    await asyncio.sleep(delay * (2 ** attempt)) # backoff
            return async_wrapper
        return decorator

@retry(max_attempts=5, delay=0.5)
async def call_llm(prompt: str) -> str: ...
```

1.3 Clases como decoradores

```
class RateLimit:
    def __init__(self, calls_per_sec: float):
        self.min_interval = 1.0 / calls_per_sec
        self._last = 0.0

    def __call__(self, func):
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            now = time.monotonic()
```

```

        wait = self.min_interval - (now - self._last)
        if wait > 0:
            await asyncio.sleep(wait)
        self._last = time.monotonic()
        return await func(*args, **kwargs)
    return wrapper

@RateLimit(calls_per_sec=10)
async def embed(text: str): ...

```

1.4 Stacking y orden de evaluación

```

# Los decoradores se aplican de abajo hacia arriba
@timer          # 3° envuelve
@retry(3)       # 2° envuelve
@RateLimit(10)  # 1° envuelve (más cercano a la función)
async def call_tool(name: str): ...

# Equivale a:
# call_tool = timer(retry(3)(RateLimit(10)(call_tool_original)))

```

■ El orden importa: si `timer` envuelve `retry`, mide también el tiempo de los reintentos. Diseña la cadena conscientemente.

1.5 Decoradores para registrar tools en MCP/LangChain

```

# Patrón registry – usado internamente por FastMCP y LangChain
_TOOL_REGISTRY: dict[str, callable] = {}

def tool(name: str | None = None, description: str = ''):
    def decorator(func):
        key = name or func.__name__
        func._tool_meta = {'name': key, 'description': description}
        _TOOL_REGISTRY[key] = func
        @functools.wraps(func)
        async def wrapper(*a, **kw):
            return await func(*a, **kw)
        return wrapper
    return decorator

@tool(description='Busca en la base de datos de turnos')
async def search_shifts(agency_id: int, date: str) -> list[dict]: ...

```

02 Pydantic v2 Validación, serialización y settings

Pydantic v2 (reescrita en Rust vía pydantic-core) es la columna vertebral de FastAPI, FastMCP y Pydantic AI. Dominarla es non-negociable para construir agentes IA robustos.

2.1 BaseModel — núcleo

```
from pydantic import BaseModel, Field, field_validator, model_validator
from typing import Annotated, Literal
from datetime import datetime

class ToolCall(BaseModel):
    model_config = {'str_strip_whitespace': True, 'validate_default': True}

    tool_name: str = Field(min_length=1, max_length=128)
    arguments: dict[str, object] = Field(default_factory=dict)
    agency_id: Annotated[int, Field(gt=0)]
    timestamp: datetime = Field(default_factory=datetime.utcnow)

    @field_validator('tool_name')
    @classmethod
    def validate_tool_name(cls, v: str) -> str:
        allowed = {'search_shifts', 'get_schedule', 'update_callshift'}
        if v not in allowed:
            raise ValueError(f'tool desconocida: {v}')
        return v

    @model_validator(mode='after')
    def check_agency_context(self) -> 'ToolCall':
        # validación cross-field
        if self.tool_name == 'update_callshift' and not self.arguments:
            raise ValueError('update_callshift requiere arguments')
        return self
```

2.2 Discriminated Unions — tipo de mensaje de agente

```
from typing import Union

class TextContent(BaseModel):
    type: Literal['text']
    text: str

class ToolUseContent(BaseModel):
    type: Literal['tool_use']
    tool_name: str
    input: dict

class ToolResultContent(BaseModel):
    type: Literal['tool_result']
    tool_use_id: str
    content: str

# Pydantic elige el modelo correcto en O(1) usando el campo 'type'
Content = Annotated[
    Union[TextContent, ToolUseContent, ToolResultContent],
    Field(discriminator='type')
]

class Message(BaseModel):
    role: Literal['user', 'assistant']
    content: list[Content]

msg = Message.model_validate({
```

```

        'role': 'assistant',
        'content': [{'type': 'tool_use', 'tool_name': 'search_shifts', 'input': {}}]
    })

```

2.3 BaseSettings — gestión de configuración

```

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file='.env',
        env_file_encoding='utf-8',
        env_nested_delimiter='__', # DATABASE__HOST -> database.host
        case_sensitive=False,
    )

    anthropic_api_key: str
    database_url: str
    redis_url: str = 'redis://localhost:6379'
    max_concurrent_tools: int = Field(default=5, gt=0)
    debug: bool = False

# Singleton thread-safe con lru_cache
from functools import lru_cache

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

```

■ pydantic-settings carga .env, variables de entorno y archivos TOML/YAML. Úsalo como única fuente de verdad para config en producción.

2.4 Serialización y JSON Schema (vital para LLMs)

```

import json

class SearchParams(BaseModel):
    """Parámetros para buscar turnos disponibles."""
    agency_id: int = Field(description='ID único de la agencia')
    date_from: str = Field(pattern=r'^\d{4}-\d{2}-\d{2}$')
    date_to: str = Field(pattern=r'^\d{4}-\d{2}-\d{2}$')
    positions: list[str] = Field(default_factory=list)

# JSON Schema para tool_use de Claude / OpenAI function calling
schema = SearchParams.model_json_schema()
print(json.dumps(schema, indent=2))

# Serializar instancia
params = SearchParams(agency_id=42, date_from='2025-01-01', date_to='2025-01-31')
params.model_dump() # dict Python
params.model_dump_json() # JSON string (Rust, muy rápido)
params.model_dump(exclude_none=True, by_alias=True) # control fino

```

03

asyncio

Concurrencia cooperativa para I/O masivo

asyncio es el modelo de concurrencia central de Python para agentes IA: todas las llamadas a LLMs, bases de datos async (asyncpg) y servidores MCP son I/O-bound y encajan perfectamente con el event loop de una sola hebra.

3.1 Coroutines, Tasks y el event loop

```
import asyncio

# coroutine – no ejecuta hasta awaited
async def fetch_llm(prompt: str) -> str:
    await asyncio.sleep(0.1) # simula latencia red
    return f'respuesta a: {prompt}'

# Task – ejecuta concurrentemente en el event loop
async def main():
    # Opción A: gather – espera TODAS
    results = await asyncio.gather(
        fetch_llm('¿qué es MCP?'),
        fetch_llm('explica asyncio'),
        fetch_llm('ventajas de Pydantic'),
    )

    # Opción B: TaskGroup (Python 3.11+) – cancela todo si una falla
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_llm('prompt 1'))
        t2 = tg.create_task(fetch_llm('prompt 2'))
    r1, r2 = t1.result(), t2.result()

asyncio.run(main())
```

3.2 Semaphore — limitar concurrencia con LLMs

```
# Evita rate-limit: máximo N llamadas simultáneas al API
sem = asyncio.Semaphore(5)

async def safe_call(prompt: str) -> str:
    async with sem:
        return await fetch_llm(prompt)

async def batch_process(prompts: list[str]) -> list[str]:
    return await asyncio.gather(*[safe_call(p) for p in prompts])
```

3.3 asyncio.Queue — patrón productor/consumer (agentes)

```
async def producer(queue: asyncio.Queue, items: list):
    for item in items:
        await queue.put(item)
    await queue.put(None) # sentinel

async def consumer(queue: asyncio.Queue, worker_id: int):
    while True:
        item = await queue.get()
        if item is None:
            queue.task_done()
            break
        result = await fetch_llm(item)
        print(f'[worker-{worker_id}] {result}')
        queue.task_done()

async def pipeline(items: list, n_workers: int = 3):
    q = asyncio.Queue(maxsize=10)
```

```
workers = [asyncio.create_task(consumer(q, i)) for i in range(n_workers)]
await producer(q, items)
# propagar sentinel a todos los workers
for _ in workers:
    await q.put(None)
await asyncio.gather(*workers)
```

3.4 Timeout y cancelación

```
# Python 3.11+ — asyncio.timeout
async def call_with_timeout(prompt: str) -> str | None:
    try:
        async with asyncio.timeout(10.0): # 10 segundos
            return await fetch_llm(prompt)
    except asyncio.TimeoutError:
        return None

# Cancelar una task externamente
task = asyncio.create_task(call_with_timeout('...'))
await asyncio.sleep(1)
task.cancel() # lanza CancelledError dentro de la coroutine
try:
    await task
except asyncio.CancelledError:
    pass # limpieza aquí
```

■ Never call `time.sleep()` dentro de una coroutine. Usa siempre `await asyncio.sleep()`. `time.sleep()` bloquea el event loop entero.

3.5 contextvars — contexto por-tarea (request ID, tracing)

```
from contextvars import ContextVar
import uuid

request_id: ContextVar[str] = ContextVar('request_id', default='')

async def handle_agent_request(payload: dict):
    token = request_id.set(str(uuid.uuid4())) # aislado por task
    try:
        await process(payload)
    finally:
        request_id.reset(token)

async def process(payload):
    rid = request_id.get()
    print(f'[{rid}] procesando {payload}') # cada task tiene su propio ID
```

■ `contextvars` es thread-safe Y task-safe. Es la forma correcta de propagar request IDs, tenant IDs o trace contexts en sistemas async multi-tenant como First Due.

04

ThreadPoolExecutor I/O-bound bloqueante en async

Cuando tienes código bloqueante (librerías síncronas, drivers JDBC, operaciones de filesystem) dentro de un contexto async, **ThreadPoolExecutor** permite ejecutarlo sin bloquear el event loop.

4.1 loop.run_in_executor — integración con asyncio

```
import asyncio
from concurrent.futures import ThreadPoolExecutor
import requests # librería síncrona

_thread_pool = ThreadPoolExecutor(max_workers=10)

async def fetch_sync_in_async(url: str) -> dict:
    loop = asyncio.get_running_loop()
    # Ejecuta la función síncrona en un hilo separado
    response = await loop.run_in_executor(
        _thread_pool,
        requests.get, # callable
        url            # args (posicionales)
    )
    return response.json()

# Para kwargs usa functools.partial
import functools
async def fetch_with_headers(url: str, headers: dict) -> dict:
    loop = asyncio.get_running_loop()
    fn = functools.partial(requests.get, url, headers=headers, timeout=30)
    resp = await loop.run_in_executor(_thread_pool, fn)
    return resp.json()
```

4.2 Uso directo con submit y as_completed

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def process_batch_sync(items: list[str]) -> list[str]:
    results = []
    with ThreadPoolExecutor(max_workers=8) as executor:
        futures = {executor.submit(heavy_io_task, item): item
                   for item in items}

        for future in as_completed(futures, timeout=30):
            original = futures[future]
            try:
                result = future.result()
                results.append(result)
            except Exception as e:
                print(f'Error en {original}: {e}')
    return results

def heavy_io_task(item: str) -> str:
    # Operación bloqueante: llamada a DB síncrona, archivo, etc.
    import time; time.sleep(0.1)
    return f'procesado: {item}'
```

4.3 contextvars con ThreadPoolExecutor

```
# Los hilos NO heredan ContextVar automáticamente — debes propagarlos
import contextvars

def run_with_context(func, *args, ctx=None):
    if ctx is not None:
        return ctx.run(func, *args) # ejecuta dentro del contexto copiado
    return func(*args)
```



```
async def safe_thread_call(item: str) -> str:
    loop = asyncio.get_running_loop()
    ctx = contextvars.copy_context() # snapshot del contexto actual
    fn = functools.partial(run_with_context, heavy_io_task, item, ctx=ctx)
    return await loop.run_in_executor(_thread_pool, fn)
```

4.4 Cuándo usar Thread vs Process vs async

Escenario	Solución	Por qué
Llamadas HTTP async (httpx, aiohttp)	asyncio.gather	I/O no bloquea el loop
Librería síncrona (requests, psycopg2)	ThreadPoolExecutor	Desbloquea el event loop
CPU-bound: parsear JSON grande	ProcessPoolExecutor	Evita el GIL
CPU-bound: ML inference local	ProcessPoolExecutor	Paralelismo real
Operaciones mixtas en pipeline	asyncio + ambos Executors	Composición flexible

05

ProcessPoolExecutor CPU-bound y paralelismo real

El GIL (Global Interpreter Lock) de CPython impide verdadero paralelismo de hilos para código Python puro. ProcessPoolExecutor lanza procesos separados, cada uno con su propio intérprete y GIL.

5.1 Uso básico

```
from concurrent.futures import ProcessPoolExecutor, as_completed
import multiprocessing

# IMPORTANTE: Las funciones deben ser picklables (definidas a nivel módulo)
def cpu_intensive(data: list[int]) -> int:
    return sum(x ** 2 for x in data)

def parallel_compute(chunks: list[list[int]]) -> list[int]:
    n_cpus = multiprocessing.cpu_count()
    with ProcessPoolExecutor(max_workers=n_cpus) as executor:
        futures = [executor.submit(cpu_intensive, chunk) for chunk in chunks]
    return [f.result() for f in as_completed(futures)]
```

5.2 Shared Memory (Python 3.8+) — sin serialización

```
from multiprocessing.shared_memory import SharedMemory
import numpy as np

def worker_with_shm(shm_name: str, shape: tuple, dtype: str) -> float:
    shm = SharedMemory(name=shm_name)
    arr = np.ndarray(shape, dtype=dtype, buffer=shm.buf)
    result = arr.mean() # operación sobre memoria compartida
    shm.close()
    return result

def main_shm():
    data = np.random.rand(10_000_000).astype('float32')
    shm = SharedMemory(create=True, size=data.nbytes)
    shared_arr = np.ndarray(data.shape, dtype=data.dtype, buffer=shm.buf)
    shared_arr[:] = data

    with ProcessPoolExecutor(max_workers=4) as ex:
        futures = [ex.submit(worker_with_shm, shm.name, data.shape, str(data.dtype))
                    for _ in range(4)]
        results = [f.result() for f in futures]

    shm.close()
    shm.unlink() # libera memoria
    return results
```

■ Serialización con pickle es el cuello de botella principal de ProcessPoolExecutor. Para arrays grandes usa SharedMemory o multiprocessing.Array.

5.3 Integración con asyncio

```
async def async_cpu_bound(data: list[int]) -> int:
    loop = asyncio.get_running_loop()
    with ProcessPoolExecutor() as pool:
        result = await loop.run_in_executor(pool, cpu_intensive, data)
    return result

# Para reutilizar el pool (no recrear en cada llamada):
_process_pool = ProcessPoolExecutor(max_workers=4)

async def reuse_pool(data: list[int]) -> int:
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(_process_pool, cpu_intensive, data)
```

6.1 Context Managers async — gestión de recursos

```
from contextlib import asynccontextmanager
from typing import AsyncGenerator
import asyncpg

@asynccontextmanager
async def db_transaction(
    pool: asyncpg.Pool
) -> AsyncGenerator[asyncpg.Connection, None]:
    async with pool.acquire() as conn:
        async with conn.transaction():
            try:
                yield conn
            except Exception:
                raise # el context manager hace rollback automáticamente

# Uso:
async def update_shift(pool, shift_id: int, data: dict):
    async with db_transaction(pool) as conn:
        await conn.execute('UPDATE shifts SET ... WHERE id=$1', shift_id)
```

6.2 Protocol — duck typing estático

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class LLMBBackend(Protocol):
    async def complete(self, prompt: str, **kwargs) -> str: ...
    async def embed(self, text: str) -> list[float]: ...

# Cualquier clase que implemente estos métodos satisface el protocolo
# sin necesidad de herencia explícita – structural subtyping
class AnthropicBackend:
    async def complete(self, prompt: str, **kwargs) -> str:
        # llamada a claude-sonnet-4-5
        ...
    async def embed(self, text: str) -> list[float]:
        ...

class MockBackend:
    async def complete(self, prompt: str, **kwargs) -> str:
        return 'mock response'
    async def embed(self, text: str) -> list[float]:
        return [0.0] * 1536

def use_backend(backend: LLMBBackend): ... # acepta cualquiera de las dos
```

6.3 Generics y TypeVar — agentes type-safe

```
from typing import TypeVar, Generic
from pydantic import BaseModel

InputT = TypeVar('InputT', bound=BaseModel)
OutputT = TypeVar('OutputT', bound=BaseModel)

class Agent(Generic[InputT, OutputT]):
    def __init__(self, llm: LLMBBackend, output_model: type[OutputT]):
        self.llm = llm
        self.output_model = output_model
```

```

    async def run(self, input_data: InputT) -> OutputT:
        prompt = self._build_prompt(input_data)
        raw = await self.llm.complete(prompt)
        return self.output_model.model_validate_json(raw)

    def _build_prompt(self, data: InputT) -> str:
        return f'Input: {data.model_dump_json()}'

# El type checker sabe exactamente qué devuelve
agent: Agent[SearchParams, ToolCall] = Agent(AnthropicBackend(), ToolCall)

```

6.4 __slots__ y dataclasses — rendimiento en hot paths

```

from dataclasses import dataclass, field

@dataclass(slots=True, frozen=True)  # Python 3.10+
class AgentMessage:
    role: str
    content: str
    metadata: dict = field(default_factory=dict, compare=False)

# slots=True: ~40% menos memoria, acceso de atributos más rápido
# frozen=True: inmutable, hashable, seguro como key de dict/set

```

El **Model Context Protocol** (Anthropic, 2024) es el estándar para exponer herramientas, recursos y plantillas de prompts a LLMs. **FastMCP** es el SDK oficial de Python — usa decoradores, Pydantic y asyncio internamente.

7.1 Servidor mínimo

```
# pip install mcp fastmcp
from fastmcp import FastMCP
from pydantic import BaseModel

mcp = FastMCP('scheduling-mcp', version='1.0.0')

class ShiftQuery(BaseModel):
    agency_id: int
    date: str

@mcp.tool()
async def search_available_shifts(query: ShiftQuery) -> list[dict]:
    """Busca turnos disponibles para una agencia en una fecha dada."""
    # Pydantic valida automáticamente los argumentos del LLM
    return await db_query(query.agency_id, query.date)

@mcp.resource('shifts://{agency_id}/schedule')
async def get_schedule(agency_id: int) -> str:
    """Expone el schedule como recurso accesible por URI."""
    data = await fetch_schedule(agency_id)
    return json.dumps(data)

@mcp.prompt()
def analyze_coverage(agency_id: int, date: str) -> str:
    """Template de prompt para análisis de cobertura."""
    return f'Analiza la cobertura de turnos para agencia {agency_id} el {date}.'

if __name__ == '__main__':
    mcp.run(transport='stdio') # Claude Desktop
    # mcp.run(transport='sse', port=8000) # HTTP/SSE para producción
```

7.2 Context y dependencias

```
from fastmcp import Context

@mcp.tool()
async def update_shift_status(
    shift_id: int,
    status: str,
    ctx: Context # inyectado automáticamente por FastMCP
) -> dict:
    """Actualiza el estado de un turno."""
    await ctx.info(f'Actualizando shift {shift_id} -> {status}')

    # Llamar a otro tool desde dentro del tool
    current = await ctx.call_tool('search_available_shifts',
                                   {'agency_id': 1, 'date': '2025-01-01'})

    return {'updated': True, 'current': current}
```

7.3 Testing de MCP tools

```
# FastMCP permite llamar tools directamente sin servidor
from fastmcp import Client
import pytest
```

```
@pytest.mark.asyncio
async def test_search_shifts():
    async with Client(mcp) as client:
        result = await client.call_tool(
            'search_available_shifts',
            {'query': {'agency_id': 42, 'date': '2025-01-15'}}
        )
        assert isinstance(result, list)
```

Framework	Fortaleza	Úsalo cuando...
Pydantic AI	Type-safety, integración nativa Pydantic v2, async first	Agentes simples a medios, LLM structured output, máximo control
LangChain	Ecosistema enorme, LCEL chains, integraciones	Prototipado rápido, muchos conectores ya hechos
LlamaIndex	RAG avanzado, indexing, query engines	Knowledge bases, búsqueda semántica, documentos
CrewAI	Multi-agente, roles, colaboración	Workflows multi-agente con roles definidos
AutoGen (Microsoft)	Conversaciones multi-agente, code exec	Agentes que generan y ejecutan código dinámicamente

8.1 Pydantic AI — agente estructurado

```
# pip install pydantic-ai
from pydantic_ai import Agent
from pydantic_ai.models.anthropic import AnthropicModel
from pydantic import BaseModel

class ShiftAnalysis(BaseModel):
    coverage_pct: float
    critical_gaps: list[str]
    recommendation: str

agent = Agent(
    AnthropicModel('claude-sonnet-4-5'),
    result_type=ShiftAnalysis,
    system_prompt='Eres un experto en planificación de turnos de emergencia.'
)

@agent.tool
async def get_shift_data(ctx, agency_id: int, date: str) -> str:
    data = await fetch_schedule(agency_id, date)
    return json.dumps(data)

async def analyze():
    result = await agent.run('Analiza cobertura agencia 42 para enero 2025')
    analysis: ShiftAnalysis = result.data # fully typed!
    return analysis
```

8.2 LangChain — LCEL chain con tools

```
from langchain_anthropic import ChatAnthropic
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate

@tool
def search_shifts(agency_id: int, date: str) -> str:
    """Busca turnos disponibles."""
    return json.dumps({'shifts': ['morning', 'evening']})

llm = ChatAnthropic(model='claude-sonnet-4-5')
tools = [search_shifts]
prompt = ChatPromptTemplate.from_messages([
    ('system', 'Eres un asistente de scheduling.'),
    ('human', '{input}'),
    ('placeholder', '{agent_scratchpad}'),
])
```

```
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = executor.invoke({'input': '¿Qué turnos hay disponibles para agencia 42?'})
```

8.3 Patrón handoff multi-agente

```
# Patrón común: router agent delega a specialist agents
class AgentRouter:
    def __init__(self):
        self.agents: dict[str, Agent] = {
            'scheduling': scheduling_agent,
            'reporting': reporting_agent,
            'compliance': compliance_agent,
        }

    async def route(self, query: str) -> str:
        # El router LLM decide qué agente usar
        domain = await self._classify(query)
        agent = self.agents.get(domain)
        if not agent:
            return 'No tengo un agente para ese dominio.'
        result = await agent.run(query)
        return result.data

    async def _classify(self, query: str) -> str:
        # Llamada liviana al LLM para clasificar
        ...
```


En sistemas multi-modelo o multi-proveedor, un **gateway LLM** centraliza routing, rate limiting, logging de costes y fallback. Nunca accedas a diferentes providers directamente desde cada agente.

9.1 LiteLLM — proxy unificado

```
# pip install litellm
import litellm

litellm.set_verbose = False

# API idéntica para todos los providers
response = await litellm.acompletion(
    model='anthropic/claude-sonnet-4-5',
    messages=[{'role': 'user', 'content': 'hola'}],
    max_tokens=1000,
)

# Fallback automático: si Anthropic falla, usa OpenAI
response = await litellm.acompletion(
    model='anthropic/claude-sonnet-4-5',
    fallbacks=['openai/gpt-4o'],
    messages=[{'role': 'user', 'content': 'hola'}],
)

# Cost tracking built-in
cost = litellm.completion_cost(completion_response=response)
print(f'Coste: ${cost:.6f}')
```

9.2 LiteLLM Proxy Server

```
# litellm_config.yaml
# model_list:
#   - model_name: fast
#     litellm_params:
#       model: anthropic/claude-haiku-4-5
#   - model_name: smart
#     litellm_params:
#       model: anthropic/claude-sonnet-4-5
# general_settings:
#   master_key: sk-my-key
#   store_model_in_db: true

# litellm --config litellm_config.yaml --port 4000

# Uso desde cualquier cliente OpenAI-compatible:
from openai import AsyncOpenAI
client = AsyncOpenAI(base_url='http://localhost:4000', api_key='sk-my-key')
resp = await client.chat.completions.create(
    model='smart',
    messages=[{'role': 'user', 'content': 'hola'}]
)
```

9.3 Helicone — observabilidad sin cambiar código

```
import anthropic

# Sólo cambias la base_url – zero code change
client = anthropic.Anthropic(
    base_url='https://anthropic.helicone.ai',
    default_headers={
        'Helicone-Auth': f'Bearer {HELICONE_API_KEY}',
        'Helicone-Property-AgencyId': str(agency_id),
    },
)
```

```

        'Helicone-Property-Feature': 'scheduling',
    }
)

# Helicone captura: latencia, tokens, coste, prompt/completion
# Dashboard: https://helicone.ai

```

9.4 Portkey — routing avanzado y guardrails

```

# pip install portkey-ai
from portkey_ai import AsyncPortkey

portkey = AsyncPortkey(
    api_key=PORTKEY_API_KEY,
    config={
        'strategy': {'mode': 'fallback'},
        'targets': [
            {'provider': 'anthropic', 'api_key': ANTHROPIC_KEY,
             'override_params': {'model': 'claude-sonnet-4-5'}},
            {'provider': 'openai', 'api_key': OPENAI_KEY,
             'override_params': {'model': 'gpt-4o'}},
        ],
        'cache': {'mode': 'semantic', 'max_age': 3600},
        'retry': {'attempts': 3},
        'guardrails': [{'id': 'no-pii'}],
    }
)

response = await portkey.chat.completions.create(
    messages=[{'role': 'user', 'content': 'hola'}]
)

```

■ Portkey añade semantic caching — respuestas similares reutilizan la misma completión. En sistemas multi-tenant como First Due puede reducir costes un 30-60%.

10.1 structlog — logging estructurado

```
# pip install structlog
import structlog

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt='iso'),
        structlog.processors.JSONRenderer(), # salida JSON para ELK/Datadog
    ]
)

log = structlog.get_logger()

async def handle_tool_call(tool_name: str, args: dict):
    structlog.contextvars.bind_contextvars(
        tool=tool_name,
        agency_id=args.get('agency_id')
    )
    log.info('tool_call_started', args=args)
    try:
        result = await dispatch_tool(tool_name, args)
        log.info('tool_call_success', result_size=len(str(result)))
        return result
    except Exception as e:
        log.error('tool_call_failed', error=str(e), exc_info=True)
        raise
    finally:
        structlog.contextvars.unbind_contextvars('tool', 'agency_id')
```

10.2 tenacity — retry robusto

```
# pip install tenacity
from tenacity import (
    retry, stop_after_attempt, wait_exponential,
    retry_if_exception_type, before_sleep_log
)
import logging

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=1, max=30),
    retry=retry_if_exception_type((anthropic.RateLimitError, anthropic.APIConnectionError)),
    before_sleep=before_sleep_log(logging.getLogger(), logging.WARNING),
    reraise=True,
)

async def resilient_llm_call(prompt: str) -> str:
    return await llm.complete(prompt)
```

10.3 pytest async — testing de agentes y MCP tools

```
# pip install pytest pytest-asyncio
# pytest.ini: asyncio_mode = auto

import pytest
from unittest.mock import AsyncMock, patch

@pytest.fixture
async def mock_llm():
```

```

    with patch('myapp.llm.complete', new_callable=AsyncMock) as mock:
        mock.return_value = '{"coverage_pct": 85.0, "critical_gaps": [], "recommendation": "OK"}'
    yield mock

@pytest.mark.asyncio
async def test_agent_structured_output(mock_llm):
    result = await agent.run('analiza cobertura agencia 42')
    assert isinstance(result.data, ShiftAnalysis)
    assert 0 <= result.data.coverage_pct <= 100
    mock_llm.assert_called_once()

```

10.4 Docker — contenedor para servidor MCP

```

# Dockerfile
FROM python:3.12-slim
WORKDIR /app

RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

COPY src/ ./src/
ENV PYTHONUNBUFFERED=1

CMD ['uv', 'run', 'python', '-m', 'src.mcp_server']

# docker-compose.yml
# services:
#   mcp-server:
#     build: .
#     ports: ['8000:8000']
#     env_file: .env
#     restart: unless-stopped

```

10.5 pyproject.toml moderno

```

[project]
name = 'scheduling-mcp'
version = '1.0.0'
requires-python = '>=3.12'
dependencies = [
    'fastmcp>=2.0',
    'pydantic>=2.7',
    'pydantic-settings>=2.3',
    'pydantic-ai>=0.0.13',
    'litellm>=1.40',
    'anthropic>=0.30',
    'asynccpg>=0.29',
    'structlog>=24.1',
    'tenacity>=8.3',
]

[dependency-groups]
dev = ['pytest>=8', 'pytest-asyncio>=0.23', 'ruff>=0.4', 'mypy>=1.10']

[tool.ruff.lint]
select = ['E', 'F', 'I', 'UP', 'ASYNC']

```

Stack recomendado para MCP + Agentes en Python

Capa	Herramienta	Por qué
------	-------------	---------

MCP Server	FastMCP 2.x	Decoradores, Pydantic nativo, stdio+SSE
Validación	Pydantic v2	Speed (Rust core), JSON Schema automático
Async I/O	asyncio + httpx	Event loop, Semaphore, TaskGroup
Threads	ThreadPoolExecutor	Librerías síncronas, DB drivers legacy
Procesos	ProcessPoolExecutor	ML inference CPU-bound local
Agente framework	Pydantic AI / LangChain	Structured output / ecosistema amplio
LLM Gateway	LiteLLM + Helicone	Multi-provider, cost tracking, fallback
Observabilidad	structlog + tenacity	JSON logs, retry robusto
Testing	pytest-asyncio + AsyncMock	Agentes y MCP tools testeables
Packaging	uv + pyproject.toml	Rápido, reproducible, moderno